Оптимизация поиска базовых чанков в системах дедупликации данных на основе сходства (SBC)

Задание 2: Сравнительный анализ подходов, архитектурные выводы и обоснование постановки задачи

1. Сравнительный анализ исследованных методов и инструментов

На основе выполненного обзора литературы проведено качественное и архитектурное сопоставление ключевых подходов к обнаружению сходства блоков и дельта-компрессии. Сравнительная характеристика сведена в табл. 1.

Таблица 1: Сравнительный анализ алгоритмов обнаружения сходства и дедупликации

Метод / Алгоритм	**Сжатие**	**CPU**	**RAM**	**FP**	**Ключевое ограничение**
Классический CDC (LBFS)	Базовое	Низкая	Мин.	Нет	Не находит квази-дубликаты
Odess (CDS Sampling)	Высокое	Средняя	Низкая	Средний	Разрастание постингов в индексе
Finesse (Feature Locality)	Высокое	Высокая	Средняя	Низкий	Накладные расходы на деление
All-Pairs Inverted Index	Среднее	Низкая	Средняя	Низкий	Требует строгого порога
Marline Palantir	**Макс.**	**Мин.**	**Низкая**	**Мин.**	**Устранено связкой эвристик**

2. Анализ узких мест существующих подходов

Анализ теоретических моделей и практических реализаций позволил выявить три фундаментальных архитектурных узких места, ограничивающих производительность систем SBC:

1. **Проблема «тяжёлого хвоста» инвертированного индекса:** При индексировании сотен тысяч версий файлов частотные фичи MinHash/Рабин

образуют списки постингов длиной в тысячи элементов. Согласно распределению Ципфа, небольшая доля наиболее популярных признаков генерирует более 80% вычислительной нагрузки при обходе индекса. Время поиска кандидата начинает деградировать квадратично относительно глубины версионной истории.

2. **Разрыв между скетч-индексом и энкодером (Size Skew):** Существующие алгоритмы скетчирования (Odess, Finesse) работают с эскизами фиксированной длины и полностью абстрагированы от реального физического размера исходных блоков данных. Алгоритм нарезки FastCDC генерирует блоки с дисперсией длины от 100 байт до 64 КБ. Если скетч-индекс находит сходство между чанком длиной 4 КБ и базой длиной 32 КБ, дельта-энкодер GDelta производит полный жадный побайтовый поиск и запускает сжатие Zstd для дельты и оригинала. Однако из-за разницы размеров дельта гарантированно не может дать эффективного коэффициента сжатия и отбрасывается по порогу ложного срабатывания (False Positive, FP). Это приводит к миллионам холостых запусков тяжелого дельта-конвейера.

3. **Паразитные дельта-цепочки (Chain Bloat):** Классическая схема SBC безусловно привязывает дельтой любой блок, если дельта сжалась хоть немного лучше оригинального блока ($ratio < 0.9$). В результате блоки, у которых дельта составляет 90–95% от размера базы, сохраняются в виде дельт. Это не даёт ощутимой экономии дискового пространства, но формирует глубокие цепочки восстановления ($C_5 \to C_4 \to C_3 \to C_2 \to C_1 \to \text{Base}$), что катастрофически снижает скорость случайного чтения при декомпрессии.

3. Обоснование выбора методов, стека технологий и архитектуры

На основе анализа узких мест сформулированы следующие архитектурные решения:

- **Обоснование языка программирования Rust:** Системы вторичного хранения и дедупликации данных функционируют в условиях жестких ограничений по латентности и пропускной способности I/O. Язык Rust обеспечивает:
  - Нулевую стоимость абстракций (zero-cost abstractions) и отсутствие сборщика мусора (GC), что исключает паузы при обработке очередей NVMe;

  - Строгие гарантии безопасности работы с памятью и многопоточностью на этапе компиляции;
  - Эффективную векторизацию (SIMD) и компиляцию в высокооптимизированный машинный код (-O3, LTO).

- **Обоснование чанкера FastCDC:** Использование FastCDC обусловлено его превосходством над классическим Rabin-CDC по скорости сканирования (в 3–10 раз быстрее за счет хеша Gear) и нормализацией длины блока вокруг среднего целевого значения, что снижает дисперсию размеров.

- **Сквозной многоуровневый фильтрующий конвейер:** Для преодоления выявленных узких мест спроектирован конвейер взаимодополняющих эвристик:

$$\text{FastCDC} \longrightarrow \text{Super-Features} \xrightarrow[\text{rare_first, max_df}]{\text{All-Pairs Prefix}} \text{Candidate} \xrightarrow[\text{Size-Skew}]{\text{Pre-Filter}} \text{GDelta} \xrightarrow[\alpha]{\text{Promote}} \text{Storage}. \quad (1)$$

  1. **All-Pairs Prefix Filtering и `rare_first`:** обход инвертированного индекса в порядке возрастания документной частоты с досрочным завершением цикла candidate generation на префиксе длины $(m - \min_overlap + 1)$.
  2. **Частотная отсечка `max_df`:** исключение списков постингов, превышающих пороговое значение длины.
  3. **Size-Skew Pre-Filter:** мгновенная блокировка вызова энкодера до запуска GDelta, если отношение длин блока и базы превышает max_size_skew (например $> 2.0..2.5$).
  4. **Min-Chunk Pre-Filter:** исключение LSH-поиска для микрочанков (< 1 КБ).
  5. **Порог α (Promote to Base):** принудительное сохранение блока как новой самостоятельной базы, если размер дельты превышает долю α от размера базы ($|Delta| \geq \alpha \cdot |Base|$), что гарантированно обрывает паразитные цепочки дельт.

4. Скорректированная постановка цели и декомпозиция задач

Проведённый сравнительный анализ обосновывает корректировку формальной цели и задач исследования:

Скорректированная цель: Разработка, математическое обоснование и экспериментальная верификация комплекса алгоритмических эвристик для скетч-индекса и дельта-конвейера платформы `marline`, обеспечивающего рост пропускной способности дедупликации более чем на 40–50% при сохранении степени сжатия в пределах 0.5–1.5% от абсолютного теоретического предела полного перебора.

Скорректированная декомпозиция задач:

1. **Задача 1. Эвристики фильтрации инвертированного скетч-индекса:** Аналитический вывод порогов Жаккара $\min_overlap = \lceil \frac{2nt}{1+t} \rceil$ и точек бифуркации; реализация обхода `rare_first`, префиксной фильтрации All-Pairs, частотной отсечки `max_df` и динамической метрики неудачных верификаций `fp_metric_threshold`.

2. **Задача 2. Пре-фильтрация кандидатов дельта-конвейера (Size-Skew и Min-Chunk):** Разработка механизмов блокировки вызова дельта-энкодера при размерном перекосе блоков ($\text{skew} > \max_size_skew$) и обхода LSH-поиска для микро-чанков ($|C| < \min_chunk_size$).

3. **Задача 3. Динамическое управление жизненным циклом баз (порог α):** Реализация алгоритма разрыва длинных дельта-цепочек через объявление блока новой базой при неэффективной дельте ($|Delta| \geq \alpha \cdot |Base|$).

4. **Задача 4. Экспериментальное исследование и кросс-датасетная валидация:** Проведение сравнительного бенчмаркинга на каноническом датасете Linux Kernel 3.5.6–3.7.6 (1.31 ГБ) и независимом валидационном датасете Linux Kernel 3.4.5–3.4.7 (1.26 ГБ), построение карт Парето и формирование эксплуатационных профилей.